

Hardening & Large / Offline Deployments

Kubernetes Production · Lesson 21

Built from the official documentation

`kubernetes-sigs/kubespray` · `docs/operations`

Learning Objectives

By the end of this lesson you will be able to:

- **Harden** a cluster via `group_vars` to CIS / **kube-bench** expectations
- **Harden workloads** with `securityContext` (non-root, drop capabilities, read-only FS) and enforce it with **Pod Security Admission**
- Turn off **anonymous-auth**, enable **audit logging** and **encryption at rest**
- Tune Kubespray for **hundreds of nodes** (forks, download-once, events etcd)
- Reserve node CPU/memory with **cgroups** (`kube_reserved` / `system_reserved`)
- Install in a fully **air-gapped / offline** environment via mirrors

All four topics are driven from `group_vars` — no code changes needed.

21.1 Security Hardening

Hardening — Where & Prerequisites

The hardening guide is a single block you add to `group_vars` (e.g. `group_vars/k8s_cluster/k8s-cluster.yml`).

- Requires **Kubernetes ≥ v1.23.6** for all features.
- Use a **recent Kubespray** — some of these vars were added recently.
- Ensure **no other vars override** the hardening settings.
- After install, **verify with kube-bench** (CIS Benchmark).

One opinionated `group_vars` block flips the cluster from "default" to "CIS-aligned". Review each value against your policy.

apiserver — AuthN / AuthZ & TLS

- `authorization_modes: ['Node', 'RBAC']`
- `remove_anonymous_access: true` — anonymous-auth **off** (safe with RBAC)
- `kube_apiserver_service_account_lookup: true`
- `kube_apiserver_request_timeout: 120s`
- `tls_min_version: VersionTLS12`
- `tls_cipher_suites:` strong ECDHE/GCM + ChaCha20 suites only

Loopback binds + RBAC + strong TLS are the foundation; the next slides add encryption, admission control and audit.

apiserver — Encryption at Rest

```
kube_encrypt_secret_data: true
kube_encryption_resources: [secrets]
kube_encryption_algorithm: "secretbox"

kube_profiling: false # disable /debug/pprof
```

- Secrets are encrypted **in etcd** rather than stored in plaintext.
- **secretbox** is the chosen encryption provider.
- Profiling endpoints are **disabled** (CIS check).

Admission Plugins & PodSecurity

```
kube_apiserver_enable_admission_plugins:  
- EventRateLimit  
- AlwaysPullImages  
- NodeRestriction  
- ResourceQuota  
- PodSecurity          # + ServiceAccount, NamespaceLifecycle, LimitRanger...  
kube_apiserver_admission_control_config_file: true  
  
kube_pod_security_use_default: true  
kube_pod_security_default_enforce: restricted # kube-system exempted
```

EventRateLimit + the admission config file throttle event floods;

PSA enforces the **restricted** profile by default.

Audit Logging

```
kubernetes_audit: true
audit_log_path: "/var/log/kube-apiserver-log.json"
audit_log_maxage: 30
audit_log_maxbackups: 10
audit_log_maxsize: 100
```

- Records every API request to a JSON log on the control plane.
- Rotation: keep **30 days**, **10 backups**, **100 MB** per file.

Controller-Manager / Scheduler / etcd

- `kube_controller_manager_bind_address: 127.0.0.1` — loopback only
- `kube_scheduler_bind_address: 127.0.0.1` — loopback only
- `kube_controller_terminated_pod_gc_threshold: 50`
- `kube_controller_feature_gates: ["RotateKubeletServerCertificate=true"]`
- `etcd_deployment_type: host` — **etcd isolated** outside the cluster

Binding control-plane components to loopback removes them from the network attack surface.

kubelet Hardening

```
kubelet_authorization_mode_webhook: true
kubelet_authentication_token_webhook: true
kubelet_rotate_certificates: true
kubelet_rotate_server_certificates: true
kube_read_only_port: 0 # disable read-only port
kubelet_protect_kernel_defaults: true
kubelet_streaming_connection_idle_timeout: "5m"
kubelet_seccomp_default: true
kubelet_systemd_hardening: true
```

Also: `kubelet_make_iptables_util_chains`, `kubelet_event_record_qps: 1`,
`kube_owner: root`, `kube_cert_group: root`.

kube-bench Alignment

These `group_vars` map directly onto CIS Kubernetes Benchmark checks:

- anonymous-auth **off**, kubelet **read-only port 0**
- **audit logging** enabled with rotation
- **encryption at rest** for secrets
- **restricted** Pod Security profile
- control-plane components on **loopback**, **profiling off**

Workflow: apply hardening → re-run `cluster.yml` → run **kube-bench** → remediate any remaining FAIL items.

21.2 Workload Hardening — securityContext

Cluster hardening vs workload hardening

21.1 locked down the **cluster** (apiserver, kubelet, PSA `restricted`).

`securityContext` locks down the **workload** — how each container **process runs on the node**:

- which **UID/GID** the process uses (`runAsUser` , `runAsNonRoot`)
- which **Linux capabilities** it keeps (`capabilities.drop/add`)
- whether it can **escalate** (`allowPrivilegeEscalation`) or **write its FS** (`readOnlyRootFilesystem`)

Orthogonal to **RBAC** (who may call the API). The apiserver's `restricted` PSA (21.1) *rejects* Pods that aren't hardened — `securityContext` is how a Pod *passes*.

Two levels + process identity

Pod-level = defaults for all containers; container-level **overrides** it.

```
spec:
  securityContext:           # Pod level
    runAsNonRoot: true      # refuse to start as root (UID 0)
    runAsUser: 1000
    fsGroup: 2000           # kubelet chowns mounted volumes to this GID
    seccompProfile: { type: RuntimeDefault }
```

Field	Level	Effect
runAsUser / runAsGroup	Pod + container	process UID / GID
runAsNonRoot	Pod + container	kubelet blocks start if it would run as root
fsGroup	Pod only	group owner of mounted volumes
capabilities	container only	add / drop Linux capabilities

Pitfall: `runAsUser` does **not** chown volumes — that's `fsGroup`'s job.

Hardened container — the recipe

```
securityContext:           # container level
  allowPrivilegeEscalation: false # sets no_new_privs
  readOnlyRootFilesystem: true  # immutable "/" — mount emptyDir for writes
  capabilities:
    drop: ["ALL"]           # drop everything...
    add:  ["NET_BIND_SERVICE"] # ...add back only what is needed
```

- `drop: ["ALL"]` then add the minimum = huge attack-surface reduction.
-  `privileged: true` and `CAP_SYS_ADMIN` **force** `allowPrivilegeEscalation` back to true — you cannot half-harden a privileged container.

securityContext ↔ Pod Security Admission

- **securityContext** = what the **developer declares** (self-imposed).
- **PSA** = what the **admin enforces** per namespace (from 21.1):

```
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted
```

- A non-hardened Pod is then **rejected** with the exact violations; a hardened Pod is **admitted**.
- Verify at runtime: `kubectl exec <pod> -- id` (non-root), `touch /` fails (read-only FS).

Together: **securityContext** hardens the Pod; **PSA guarantees** no un-hardened Pod is admitted.

21.3 Large Deployments

Scaling Ansible

```
# 200 nodes: parallelise and extend timeouts
ansible-playbook -i inventory/mycluster/hosts.yaml \
  cluster.yml --forks=50 --timeout=600
```

```
download_run_once: true    # download images/binaries once, push to nodes
retry_stagger: 60         # spread retries so they don't hammer the CP
```

`download_run_once` is the single biggest scale win — nodes stop each fetching from the internet.

Scaling the Cluster

- `etcd_events_cluster_setup: true` — store **events in a dedicated etcd**, so event churn doesn't overload the main etcd.
- Add `calico_rr` route-reflector nodes for faster recovery with Calico/Canal.
- Lengthen reliability timeouts so transient pressure doesn't evict nodes:
 - `kube_controller_node_monitor_grace_period`
 - `kube_apiserver_pod_eviction_not_ready_timeout_seconds`
 - `kube_apiserver_pod_eviction_unreachable_timeout_seconds`

Separating the events store is the key etcd protection at scale.

Resource Sizing & CoreDNS

- Pattern: `foo_cpu_limit` / `foo_cpu_requests`, `foo_memory_limit` / `foo_memory_requests` — **limits must be $\geq$ requests**.
- Memory in `M`; CPU drops the `m` suffix.
- **CoreDNS:** `dns_replicas`, `dns_cpu_limit` / `dns_cpu_requests`, `dns_memory_limit` / `dns_memory_requests`.
- Size the network for the node/pod count:
`kube_network_node_prefix`, `kube_service_addresses`, `kube_pods_subnet`.

Scale CoreDNS replicas and resources with the cluster — DNS is a common bottleneck.

21.4 cgroups — Node Reservation

Node Allocatable

$\text{Allocatable} = \text{Capacity} - \text{kube_reserved} - \text{system_reserved} - \text{eviction}$

- `kubelet_enforce_node_allocatable` selects what is enforced:
 - `"pods"`
 - `"pods, kube-reserved"`
 - `"pods, kube-reserved, system-reserved"`
- **kube-reserved** = kubelet + container engine.
- **system-reserved** = OS daemons (sshd, udev...).

Reserving resources keeps the **kubelet responsive** under node pressure.

kube_reserved / system_reserved

```
kube_reserved: true
kube_reserved_cgroups_for_service_slice: kube.slice
kube_reserved_cgroups: "/{{ kube_reserved_cgroups_for_service_slice }}"
kube_memory_reserved: 256Mi
kube_cpu_reserved: 100m

system_reserved: true
system_reserved_cgroups_for_service_slice: system.slice
system_memory_reserved: 512Mi
system_cpu_reserved: 500m
```

Optional: `*_ephemeral_storage_reserved`, `*_pid_reserved` for disk/PID caps.

21.5 Offline / Air-Gapped

Offline — Hosts & Mirrors

Mirror everything Kubespray would pull, then point `group_vars` at it.

Reference: `inventory/sample/group_vars/all/offline.yml`.

- `registry_host` / `registry_addr` — internal container image registry
- `files_repo` — HTTP server for binaries / zips
- `yum_repo` / `debian_repo` / `ubuntu_repo` — internal OS package mirrors
- Helper: `contrib/offline/generate_list.sh` lists everything to mirror

Three classes of artifact: **images**, **binaries**, **OS packages** — each gets its own mirror.

Image & Binary Repo Overrides

```
kube_image_repo: "{{ registry_host }}"
gcr_image_repo: "{{ registry_host }}"
quay_image_repo: "{{ registry_host }}"
github_image_repo: "{{ registry_host }}"

github_url: "{{ files_repo }}/github.com"
dl_k8s_io_url: "{{ files_repo }}/dl.k8s.io"
storage_googleapis_url: "{{ files_repo }}/storage.googleapis.com"
get_helm_url: "{{ files_repo }}/get.helm.sh"
```

All upstream image registries and download URLs are redirected to the mirror.

OS Packages & Containerd Mirror

```
docker_rh_repo_base_url: "{{ yum_repo }}/docker-ce/$releasever/$basearch"
containerd_ubuntu_repo_base_url: "{{ ubuntu_repo }}/containerd"

containerd_registries_mirrors:
  - prefix: "{{ registry_addr }}"
    mirrors:
      - host: "{{ registry_host }}"
        capabilities: ["pull", "resolve"]
        skip_verify: true
```

Auth via `files_repo_user/pass` and `registry_user/pass` (use Ansible Vault).

Key Takeaways

- **Hardening** = one `group_vars` block: anonymous-auth off, audit on, encryption at rest, restricted PSA, kubelet locked down — then **kube-bench**.
- **Workload hardening** = `securityContext` (`runAsNonRoot` , drop `ALL` caps, `readOnlyRootFilesystem` , `allowPrivilegeEscalation: false`); **PSA** enforces it per namespace (`pod-security.kubernetes.io/enforce=restricted`).
- **Scale** = `download_run_once` , more **forks/timeout**, a **separate events etcd**, longer node-monitor/eviction timeouts.
- **cgroups** reserve CPU/mem/PIDs for kubelet (`kube_reserved`) and the OS (`system_reserved`) so workloads can't starve the node.
- **Offline** = redirect images (`*_image_repo`), binaries (`*_url`) and OS packages (`*_repo`) to internal mirrors via `offline.yml`.

End of Lesson 21

Next: Lesson 22 — Cloud & OpenStack

`kube-bench` · `download_run_once` · `kube_reserved` · `offline.yml`